

Previous-Year Question Bank

Every question inside the syllabus of the two companion decks — gathered from twelve NIT Silchar exam papers (2015–2024), the course’s own lecture slides, and checked against the reference handbook — organised topic-wise and matched back to its source.

41

question cards, topic-wise

7

syllabus topics covered

13

sources: 12 exam papers +
lecture slides

THE FILTER APPLIED

In scope: Arrays · Linked Lists & Skip Lists · Stacks · Queues & Priority Queues · Binary Search Trees · AVL Trees · Red-Black Trees · Splay Trees — exactly the two decks’ coverage.

Out of scope (see the appendix for the full list): heaps of every kind, hashing, B-trees, spatial trees (quad/KD/R/H), tries & suffix structures, PQ-trees, BSP-trees, DCEL, disjoint-sets, graph algorithms, and abstract amortized-analysis proofs.

How this question bank was built

Method

- Three kinds of source material were checked: twelve previous-year NIT Silchar "Advanced (Algorithms &) Data Structure" exam papers (CS-1502 / CS-5102 / CS5102, 2015–2024), the course's own 225-slide lecture deck (Dr. Saroj Kr Biswas), and the 1,321-page reference handbook the lecture deck is grounded in. Every question and worked example in the first two was read individually; the handbook was checked structurally (see below).
- Each individual question (down to lettered sub-parts) was checked against the exact topic list of the two reference decks built for this course: "Intro to Trees" (really Fundamentals → Arrays → Linked Lists → Stacks → Queues) and "Trees to Splay Trees" (Tree basics → Binary trees & traversals → BST → AVL → Red-Black → Splay).
- A question was kept only if its core topic is taught in one of those two decks. Questions that mix an in-scope and an out-of-scope sub-part (e.g. a priority-queue question that drifts into heap-array indexing) were split — only the in-scope portion was kept, and a note says so.
- Every kept question carries its exact source, its original mark split (where the source gave one — the lecture slides' worked examples are ungraded, so those are marked "—"), and — where the question depends on a given diagram — that diagram, redrawn from the scan. Two colour-only exam diagrams (RB-5/RB-6 and RB-8) could not be transcribed with full confidence and are flagged rather than guessed at.
- Several of the lecture slides' worked examples (BST-2, BST-3, AVL-5, AVL-6, SPLAY-8, SPLAY-9, RB-14) are the exact same examples already taught step-by-step inside the two decks. They are included anyway for completeness, each flagged with a note saying so, rather than silently dropped.
- Duplicate scans of the same exam page (several were photographed twice) were merged into one source entry each.

The handbook: checked, nothing usable found

The 1,321-page reference "Handbook of Data Structures and Applications" was also checked. It is a narrative research reference, not a textbook: it has no end-of-chapter exercise sets anywhere in it. A full-text search turned up only five incidental "left as an exercise to the reader" remarks in the entire book, and all five sit in out-of-scope chapters (compressed quadtrees, randomized/treap dictionaries, suffix-tree right-equivalence classes) — so nothing from it qualified for this question bank.

What "in scope" means here

Arrays (incl. dynamic/elastic arrays & amortized growth cost) · Linked Lists, incl. doubly/circular/multilevel & Skip Lists · Stacks & their applications · Queues, incl. circular/deque/Priority Queue · Binary Search Trees · AVL Trees (rotations, insertion, deletion) · Red-Black Trees (properties, insertion, deletion, colouring) · Splay Trees (zig/zig-zig/zig-zag, amortized cost).

Sources, and where each topic lives

Topic index

1. Arrays	1 card(s)
2. Linked Lists & Skip Lists	1 card(s)
3. Queues & Priority Queues	1 card(s)
4. Binary Search Trees	4 card(s)
5. AVL Trees	9 card(s)
6. Red-Black Trees	14 card(s)
7. Splay Trees	11 card(s)

Source papers used

2015 — CS-1502

4 in-scope questions (BST delete, RB features & insertion, splay access)

2017 (Sept) — CS-1502

no in-scope questions (heap-array proofs, amortized analysis, B-tree height, Fibonacci heap)

2018 (Nov) — CS-1502

no in-scope questions (cache-aware structures, hashing, suffix trees, KD/quad/R/H-trees, Fibonacci heap)

2024 (Oct) — CS-5102

5 in-scope questions (RB properties, RB-vs-AVL, RB validity check, BST-from-colours, AVL insert)

2016 (Sept) — CS-1502

no in-scope questions (graph MST, B-trees, binomial/Fibonacci heaps, disjoint-sets, amortized-cost proof)

2018 (Sept) — CS-1502

2 in-scope questions (skip lists, elastic/dynamic arrays)

2022 — CS-5102

7 in-scope questions (AVL figure + delete + rebalance-to-balanced, splay access/delete/top-down-splay)

2024 (Dec) — CS5102

1 in-scope question (RB-tree insertion)

lecture — 225 slides

13 in-scope worked examples (BST/AVL/RB/Splay construct, delete & search prompts embedded in the teaching flow — 7 of these are the same examples the deck already teaches; 6 are additional)

2016 (Dec) — CS-1502

2 in-scope questions (splay-vs-BST, RB-tree colouring by black-height)

2017 (Nov-Dec) — CS-1502

no in-scope questions (tries, suffix trees/arrays, PQ-trees, BSP-trees, R-trees)

2024 (Sept) — CS5102

4 in-scope questions (AVL word-insert, splay search, RB pros/cons, RB symbol-insert)

undated — CS-1502-style

2 in-scope questions (RB-tree build-up by insertion, RB true/false)

Arrays

ARR-1

Sept 2018 Mid-Sem

2+2 MARKS

Devise an array data structure which provides elasticity. Your array data structure must support grow and shrink operations. The grow operation should not cause a segmentation fault. Analyse the amortized cost of your data structure.

Maps to "Dynamic (growable) arrays" and Practice Bank Q9 (amortized cost of dynamic-array growth).

Linked Lists & Skip Lists

LL-1

Sept 2018 Mid-Sem

2 MARKS

What is the significance of the randomized Skip List? Derive the time complexity to search a key.

Maps to "Skip lists: randomized levels for $O(\log n)$ average search."

Queues & Priority Queues

PQ-1

Sept 2024 Mid-Sem

1 MARKS

What are the applications of a priority queue?

The source question also asks how to implement a PQ as an array-based heap and how to find a parent/child from an array index — that half rests on heap mechanics, which is outside this syllabus, so only the in-scope applications part is kept here.

Binary Search Trees

BST-1

2015 End-Sem

5 MARKS

The algorithm below deletes an item from a Binary Search Tree. Write the inner meaning of every algorithmic statement alongside each line, so the logic is completely clear (source lines need not be copied into your answer).

Tree-Delete(T, z)

i. if $\text{left}[z] = \text{NIL}$ or $\text{right}[z] = \text{NIL}$ // left = left child, right = right child

ii. then $y \leftarrow z$

iii. else $y \leftarrow \text{Tree-Successor}(z)$

iv. if $\text{left}[y] \neq \text{NIL}$

v. then $x \leftarrow \text{left}[y]$

vi. else $x \leftarrow \text{right}[y]$

vii. if $x \neq \text{NIL}$

viii. then $p[x] \leftarrow p[y]$

ix. if $p[y] = \text{NIL}$

x. then $\text{root}[T] \leftarrow x$

xi. else if $y = \text{left}[p[y]]$

xii. then $\text{left}[p[y]] \leftarrow x$

xiii. then $\text{right}[p[y]] \leftarrow x$

xiv. if $y \neq x$

xv. then $\text{key}[z] \leftarrow \text{key}[y]$

xvi. // If y has other fields, copy them too.

xvii. return y

Maps to "Deleting from a BST: leaf and one-child cases" / "two children."

Binary Search Trees (continued)

BST-2

[Lecture PPT](#)[— MARKS](#)

Construct a Binary Search Tree (BST) for the following sequence of numbers: 50, 70, 60, 20, 90, 10, 40, 100.

This exact worked example from the lecture slides is already taught step-by-step in the Trees deck's BST section — included here for completeness.

BST-3

[Lecture PPT](#)[— MARKS](#)

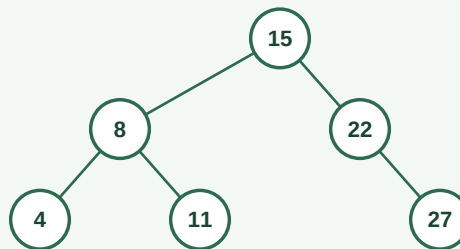
Create a Binary Search Tree using the following data elements: 45, 39, 56, 12, 34, 78, 32, 10, 89, 54, 67, 81.

Also already taught in the deck as the BST section's "second example: 12 keys."

BST-4

[Lecture PPT](#)[— MARKS](#)

In the Binary Search Tree shown below, trace a search for the present value 11, then a search for the missing value 25 — show the comparison made at every node.



AVL Trees

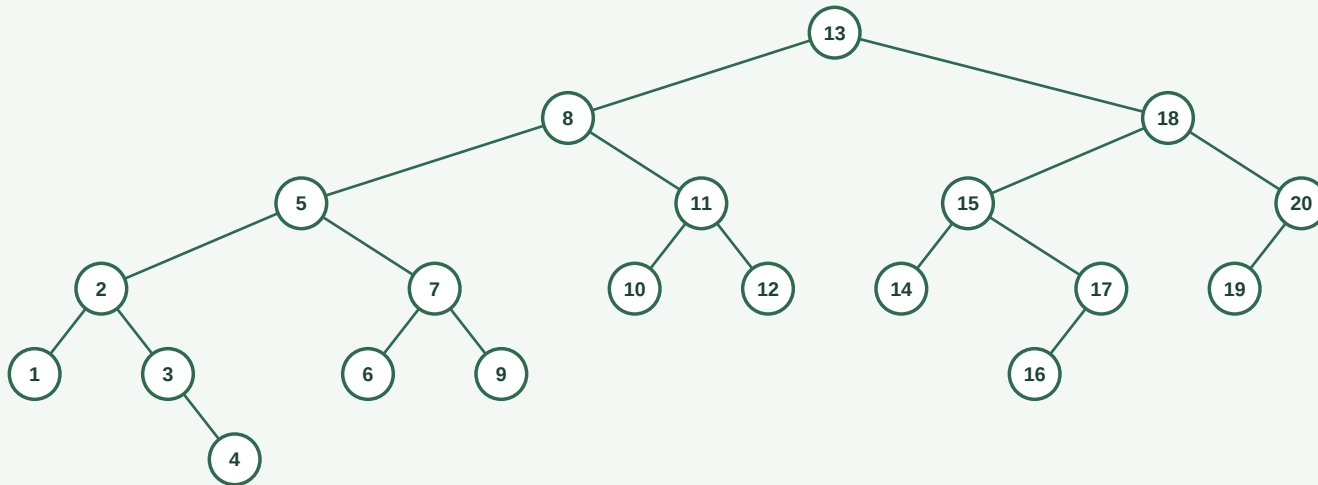
AVL-1

Dec 2022 End-Sem

2+3+5 MARKS

Consider the AVL tree shown below.

- Draw the tree again, indicating the balance factor associated with each node.
- Show the tree that results from the operation `delete(19)`, after all rebalancing has completed.
- Derive an algorithm that, using only single left- and right-rotations, converts this tree into a perfectly balanced complete binary tree (target shape shown alongside).



Given AVL tree (20 nodes). Target shape for part (c) is shown in AVL-1b below.

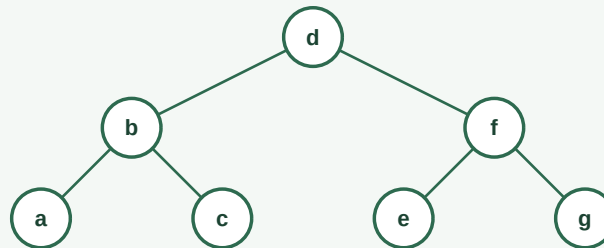
AVL Trees (continued)

AVL-1b

Dec 2022 End-Sem

— (part of AVL-1c) MARKS

Target shape for AVL-1, part (c) — the perfectly balanced complete binary tree the rotations must produce (letters stand for the 7 keys nearest the tree's centre).



AVL-2

Dec 2022 End-Sem

5 MARKS

You have a valid AVL tree with n nodes. You insert two keys — one smaller than every key in the tree and the other larger than every key in the tree — but you perform no rebalancing after these insertions. True or False: the resulting tree is still a valid AVL tree. Briefly explain your answer.

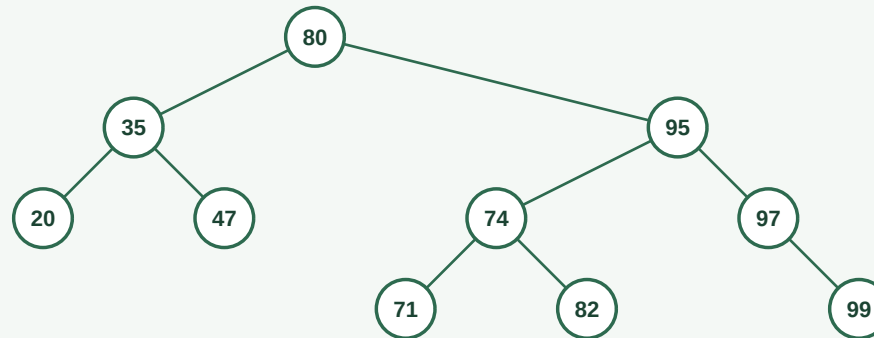
AVL Trees (continued)

AVL-3

Oct 2024 Re-exam

3 MARKS

Given the AVL tree shown below (Fig-2), insert data 72 into it. Show all steps — no short-cuts allowed.

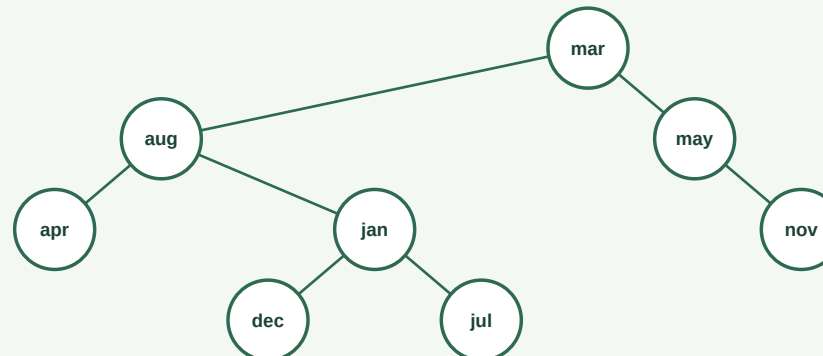


AVL-4

Sept 2024 Mid-Sem

5 MARKS

Insert the word 'feb' into the AVL tree shown below (Fig-1). Now, in the resultant tree, insert the word 'jun'. Each word is a variable name; precedence between two names is decided letter-by-letter, comparing the 1st letters first, then the 2nd if there is a tie, and so on (e.g. 'abs' < 'big', and 'zebra' > 'tiger' since z > t). Wherever an RL or LR rotation is required, show all the elementary rotations explicitly — short-cuts are not allowed.



AVL Trees (continued)

AVL-5

[Lecture PPT](#)[— MARKS](#)

Construct an AVL tree by inserting the following elements in the given order: 63, 9, 19, 27, 18, 108, 99, 81.

Already taught in the deck as "AVL example 1."

AVL-6

[Lecture PPT](#)[— MARKS](#)

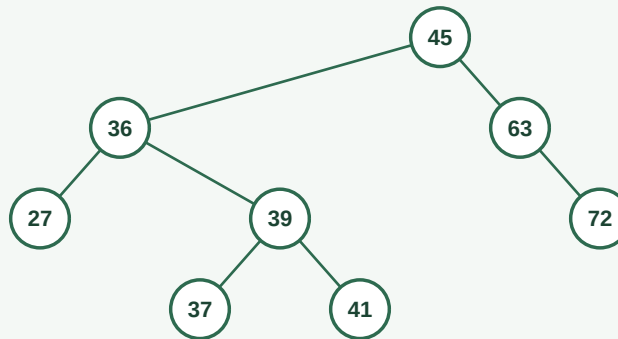
Construct an AVL tree by inserting the following elements in the given order: 30, 20, 10, 40, 50, 25.

Already taught in the deck as "AVL example 2."

AVL-7

[Lecture PPT](#)[— MARKS](#)

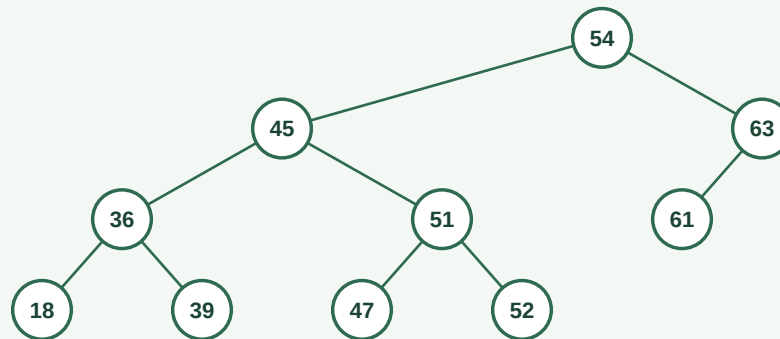
Consider the AVL tree shown below and delete 72 from it.



AVL Trees (continued)

AVL-8[Lecture PPT](#)[— MARKS](#)

Delete nodes 52, 36, and 61 (in that order) from the AVL tree shown below.



Red-Black Trees

RB-1

2015 End-Sem

2 MARKS

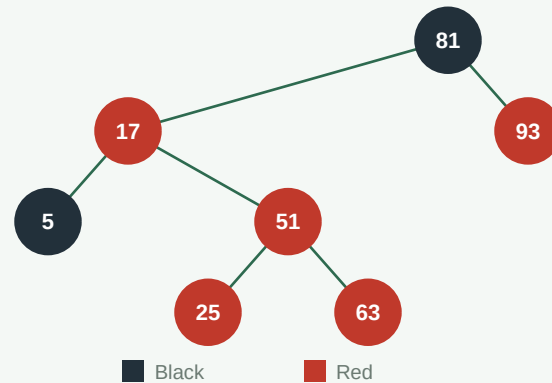
Write four important features of a Red-Black tree.

RB-2

2015 End-Sem

4 MARKS

Insert data 26 into the given Red-Black tree below. Mention the case number (case-1, case-2, etc.) while showing the different diagrams needed to explain all the required steps, and show the resultant RBT.



RB-3

Oct 2024 Re-exam

1.5 MARKS

Write down all the properties of a Red-Black Tree (RBT).

Red-Black Trees (continued)

RB-4

Oct 2024 Re-exam

1.5 MARKS

Compare and contrast an RB tree and an AVL tree.

RB-5

Oct 2024 Re-exam

1 MARKS

The source figure (Fig-1) shows only the colours of the nodes of a probable RB tree, without any data. Is the tree shown in Fig-1 a Red-Black tree in the real sense? Show all reasons to justify your answer.

Fig-1 is a colour-only 15-node tree (no keys); reproduce it from your copy of the Oct 2024 re-exam paper before attempting this — it was not redrawn here since the exact colour-by-colour layout could not be verified with full confidence from the scan.

RB-6

Oct 2024 Re-exam

2 MARKS

Draw only the structure of the tree shown in Fig-1 (see RB-5), without colours, and fill the position of each node with some two-digit number so that it satisfies the Binary Search Tree property. (You need not consider the colour of the nodes.)

RB-7

Sept 2024 Mid-Sem

1 MARKS

What is the advantage and disadvantage of an RB tree over an AVL tree?

Red-Black Trees (continued)

RB-8

Sept 2024 Mid-Sem

7 MARKS

Insert the character 'Y' into the Red-Black tree shown below (Fig-2). Consider the precedence of the symbols (smallest to largest) as $\Sigma < \Phi < \beta < \Delta < \epsilon < \pi < \psi < \mathcal{E} < [\text{reversed-E}] < \Upsilon < \Lambda < \tau < \Omega < \Theta$. Symbols marked with a superscript 'R' are Red nodes (5 Red nodes in the given figure); all others are Black.

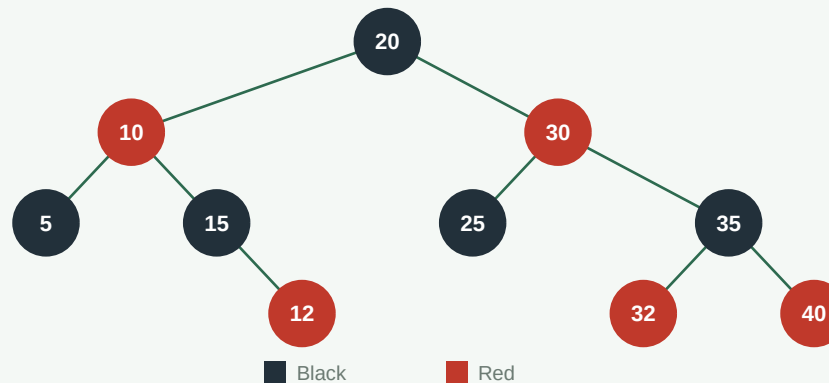
Fig-2 is a 12-node RB tree keyed on Greek/mathematical symbols rather than numbers; reproduce it from your copy of the Sept 2024 mid-sem paper — the exact colour-by-colour layout could not be pinned down with full confidence from the scan to redraw it here.

RB-9

Dec 2024 End-Sem

5 MARKS

Insert 50 into the RB tree shown below (Fig-1). Before every step, write all the actions performed along with the rule(s) applied, in short.



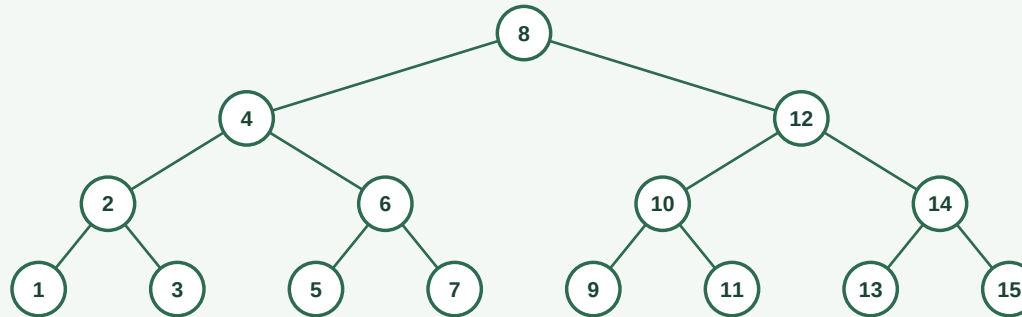
Red-Black Trees (continued)

RB-10

Dec 2016 End-Sem

3 MARKS

Draw the complete binary search tree of height 3 on the keys $\{1, 2, \dots, 15\}$. Add the NIL leaves, and colour the nodes in three different ways such that the black-heights of the resulting red-black trees are 2, 3, and 4 respectively.



Shape above is the (derivable) uncoloured base tree the question describes — not given in the source, but fully determined by it. Colouring it three ways is the task.

RB-11

PYQ excerpt (undated)

6 MARKS

Show the red-black tree that results after each of the integer keys 21, 32, 64, 75, and 15 are inserted, in that order, into an initially empty red-black tree. Clearly show the tree that results after each insertion (indicating the colour of each node), and make clear any rotations that must be performed.

Source page carries no visible exam header; reproduced verbatim from the scanned excerpt.

Red-Black Trees (continued)

RB-12

PYQ excerpt (undated)

2+2 MARKS

Indicate for each of the following statements whether it is true or false. Justify your answers to get credit.

- (i) The subtree of the root of a red-black tree is always itself a red-black tree.
- (ii) The sibling of a null child reference in a red-black tree is either another null child reference or a red node.

Continues RB-11 on the same source page.

RB-13

Lecture PPT

— MARKS

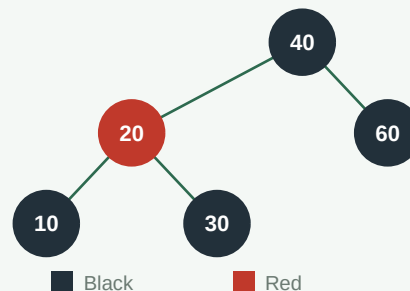
Build a Red-Black tree by inserting the numbers 10, 20, 30, and 15, one by one. Show the tree after each insertion, and note which case (recolour or rotation) each one triggers.

RB-14

Lecture PPT

— MARKS

The tree below is mid-deletion — sibling W is Black with a Red right child (Case 4 of the deletion fix-up). Delete node 30 from it and show the fully rebalanced result.



Already taught in the deck as "Deletion example 1: delete 30."

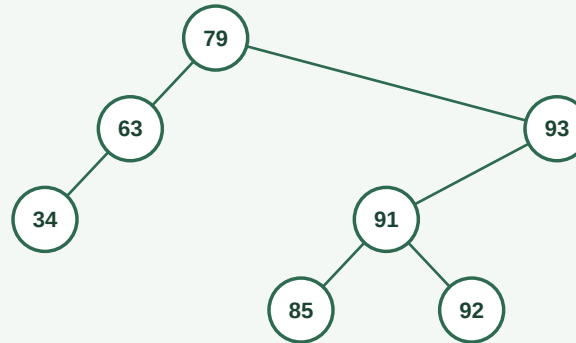
Splay Trees

SPLAY-1

2015 End-Sem

1+1+2 MARKS

What are splay operations? When is it required to be done? If data 91 is accessed from the BST shown below, what operations will be performed? Also show the final tree after splaying — show all steps.

**SPLAY-2**

Dec 2022 End-Sem

5 MARKS

You have a splay tree containing n nodes, for some large value n . Let x and y be two keys in the dictionary. You repeat the operations $\text{splay}(x)$ followed by $\text{splay}(y)$, m times, where m is much greater than n . What is the worst-case time complexity for all of these operations together? Briefly explain.

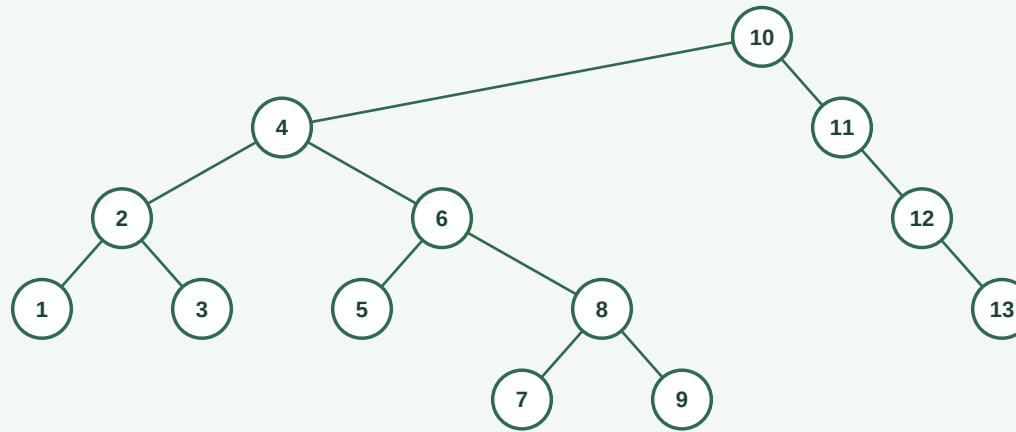
Splay Trees (continued)

SPLAY-3

Dec 2022 End-Sem

3 MARKS

Show the result of accessing the keys 3, 9, 1, 5 — in that order — in the splay tree shown below.



SPLAY-4

Dec 2022 End-Sem

5 MARKS

Continuing from SPLAY-3: show the result of deleting the element with key 6 in the resulting splay tree.

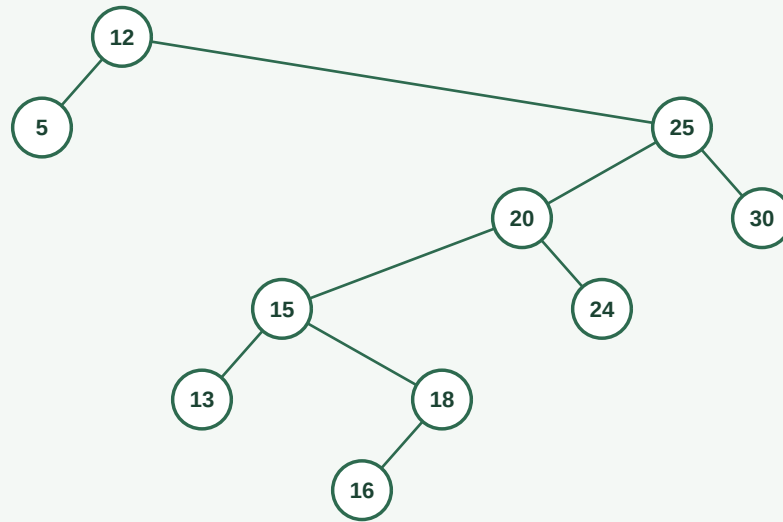
Splay Trees (continued)

SPLAY-5

Dec 2022 End-Sem

4 MARKS

Show the top-down splaying of the tree below to access 19 in the tree.



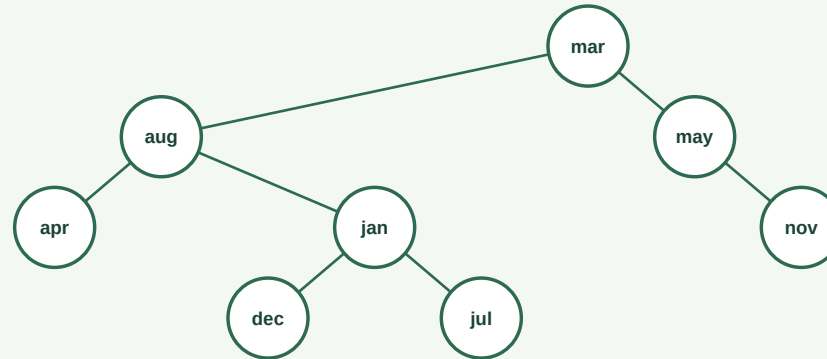
Splay Trees (continued)

SPLAY-6

Sept 2024 Mid-Sem

4 MARKS

Now treat the tree shown below (the same Fig-1 used for AVL-4) as a splay tree. Search for data 'dec' in the given tree, and show the tree that results after splaying.



SPLAY-7

Dec 2016 End-Sem

— (part of a 2×3 question) MARKS

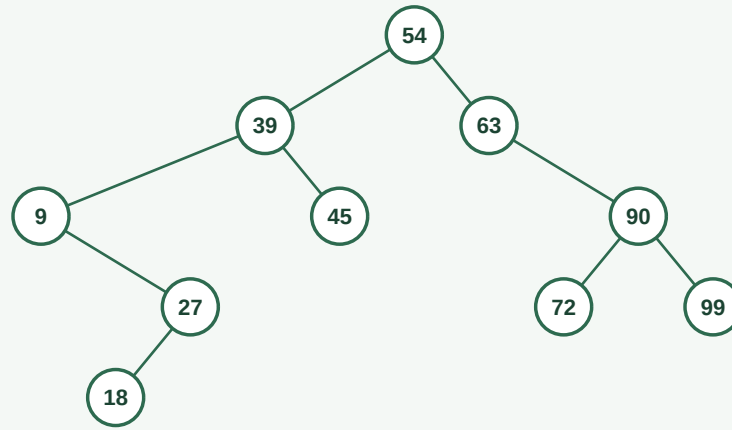
Discuss the advantage of splay trees over binary search trees.

Splay Trees (continued)

SPLAY-8

[Lecture PPT](#)[— MARKS](#)

Consider the splay tree shown below. Observe the change in its structure when 81 is added to it.

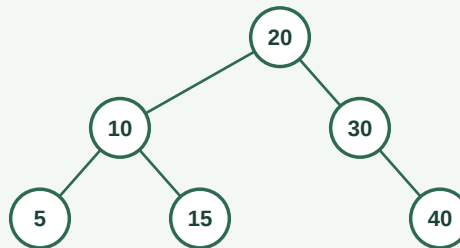


Already taught in the deck, cited there as "lecture slide 213."

SPLAY-9

[Lecture PPT](#)[— MARKS](#)

Delete the element 20 from the splay tree shown below.



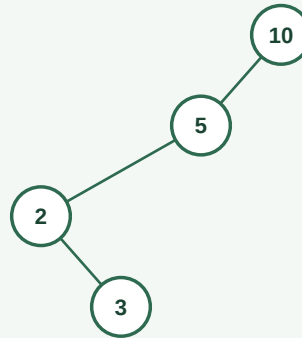
Already taught in the deck, cited there as "lecture slides 214-215."

Splay Trees (continued)

SPLAY-10

[Lecture PPT](#)[— MARKS](#)

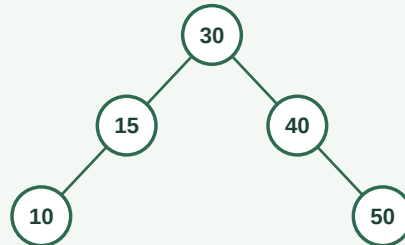
Trace a search operation for Key = 3 in the Binary Search Tree shown below, and show the tree that results after splaying.



SPLAY-11

[Lecture PPT](#)[— MARKS](#)

Trace a search operation for Key = 45 in the Binary Search Tree shown below (the key is not present), and show the tree that results after splaying the last node visited.



Topics intentionally excluded

These clusters recur across the twelve source papers but sit outside both reference decks, so every question built on them was left out of this question bank.

Heaps

binary / binomial / Fibonacci / interval / max-min heaps, heapify ("adjust"), heap-array tracing

Hashing

collision resolution (chaining, linear/quadratic probing, double hashing), bloom filters, universal hashing / MACs

B-Trees

B-tree construction, insertion, deletion, height bounds, node structure

Spatial trees

quadtrees (point/PR/region), KD-trees, R-trees / R+-trees / H-trees, range trees, fractional cascading

String / prefix structures

tries, Patricia trees, de la Briandais trees, suffix trees & arrays (Ukkonen's method)

Other advanced trees

PQ-trees, BSP-trees, doubly-connected edge lists (DCEL)

Sets & graphs

disjoint-set forests (union-find), Prim's MST, general graph algorithms

Formal amortized-analysis proofs

k-bit counter / aggregate-method proofs argued in the abstract (as opposed to the concrete dynamic-array case, which is in scope)

Misc. algorithmic puzzles

DNA-clone ordering, cache-memory design for sorted-data validation, external sorting under a fixed RAM budget, the abstract INSERT/DELETE-LARGER-HALF design problem